Project 3: Barcode & QR Code Scanner

Introduction to Computer Vision — ESIN, UIR — Spring 2026

Akoudad Karim | Student ID: 126406

1. Introduction

Barcodes and QR codes are ubiquitous in modern applications — from retail logistics to contactless payments and document authentication. This project develops a complete computer vision pipeline that detects, localizes, and decodes both 1D barcodes and 2D QR codes from still images. The solution is implemented entirely using classical image processing techniques covered in the course, with no neural network training involved in the core detection pipeline.

The system is delivered as (1) a clean, modular Jupyter notebook runnable on Google Colab, and (2) a Gradio-based web application that provides an interactive GUI for real-time scanning.

2. Approach and Pipeline

The detection pipeline consists of seven sequential stages, each grounded in a concept from the course curriculum. The full pipeline is summarized below:

Step	Operation	Course Concept
1	Grayscale conversion	Lecture 2 — Image representation
2	Gaussian blur (5×5 kernel)	Lecture 2 — Linear filtering, convolution
3	Morphological gradient (dilate − erode)	Lecture 4 — Non-linear filtering
4	Otsu binarization	Lecture 4 — Non-linear filtering
5	Morphological close (21×7 kernel)	Lecture 4 — Non-linear filtering
6	Contour analysis → bounding regions	Lecture 9 — Boundary detection
7	Multi-variant decoding (pyzbar)	Lectures 2 & 4 — filtering variants

2.1 Preprocessing (Steps 1–2)

The input image is first converted to grayscale, reducing the 3-channel BGR tensor to a single intensity channel. A 5×5 Gaussian kernel is then convolved with the image to suppress high-frequency noise before differentiation. This follows directly from Lecture 2 (Linear Filtering): the Gaussian is a linear shift-invariant filter, and smoothing before edge-like operations is a fundamental principle to avoid noise amplification.

2.2 Morphological Pipeline (Steps 3–5)

The morphological gradient — defined as the pixel-wise difference between the dilation and erosion of the blurred image — produces an edge-like response that emphasizes the high-contrast transitions characteristic of barcode stripe patterns. A wide rectangular structuring element (9×3) is chosen to preferentially highlight horizontal bar structures.

Otsu's thresholding algorithm automatically selects the optimal global threshold by minimizing intra-class intensity variance, producing a clean binary image without manual parameter tuning. A subsequent morphological close operation (21×7 kernel) bridges the gaps between individual barcode bars, merging them into solid rectangular blobs that are reliably detectable as contours.

2.3 Localization via Contour Analysis (Step 6)

Contours are extracted from the binary mask using cv2.findContours with RETR_EXTERNAL and CHAIN_APPROX_SIMPLE flags. Each contour with area above a noise threshold (1500 px²) yields an axis-aligned bounding rectangle that marks a candidate barcode or QR region. This directly applies the boundary detection and contour analysis concepts from Lecture 9.

2.4 Multi-Variant Decoding (Step 7)

The pyzbar library (a wrapper around the zbar C library) performs the actual symbol decoding. To maximize robustness under varying illumination and contrast conditions, decoding is attempted on five preprocessed variants of the image: raw BGR, grayscale, adaptive threshold, inverted adaptive threshold, and a sharpened version obtained by convolving with the kernel $K = [[0,-1,0],[-1,5,-1],[0,-1,0]]$. This sharpening filter is a direct application of linear convolution from Lecture 2. Results are deduplicated by (data, type) pair.

3. Challenges and Solutions

Native library dependency (libzbar0). pyzbar requires the native zbar shared library which is not pre-installed on Google Colab. Solved by adding an apt-get install libzbar0 call at the start of the notebook's setup cell.

ipywidgets API version mismatch. The FileUpload widget changed its internal data structure between ipywidgets v7 and v8. The upload handler was written to detect and handle both formats, ensuring compatibility across Colab runtime versions.

Decode rate on real-world images. Barcodes captured under suboptimal conditions (glare, rotation, low contrast) often fail to decode from the raw image alone. The multi-variant preprocessing strategy (adaptive thresholding, sharpening) significantly improves decode rate by presenting the decoder with multiple image representations.

Morphological kernel tuning. The structuring element dimensions for the close operation required careful tuning: too narrow and barcode bars remain disconnected; too wide and adjacent codes merge. A 21×7 kernel was found to work reliably across the test dataset.

4. Results

The pipeline was evaluated on three synthetic test images and several real-world photographs uploaded via the interactive Colab widget. All synthetic images were decoded correctly with 100% accuracy. The multi-variant decoding strategy proved essential for real-world images where the raw image alone often failed to decode.

Image	Code Type	Decoded Value	Status
qr_test.png	QRCODE	https://uir.ac.ma	**Correct**
barcode_test.png	EAN13	5901234123457	**Correct**
combined_test.png	QRCODE + EAN13	Both codes	**Correct**

The pipeline steps are visualized in the notebook, showing the transformation from raw input through grayscale, Gaussian blur, morphological gradient, Otsu binary mask, morphological close, and final annotated output. The preprocessing gallery confirms that the morphological close correctly merges barcode stripes into solid blobs prior to contour extraction.

5. Conclusion

This project demonstrates that robust barcode and QR code detection can be achieved entirely through classical computer vision — without any neural network training. The pipeline integrates linear filtering (Gaussian blur, sharpening convolution), non-linear morphological operations, Otsu thresholding, and contour-based boundary detection — concepts directly drawn from the course curriculum. The multi-variant decoding strategy provides practical robustness against real-world imaging conditions.

GitHub Repository: https://github.com/akoudad/barcode-qr-scanner